

aidev

서로를 보완하는 에이전트로 하는 포트폴리오 규모의 자율 소프트웨어 유지보수

aidev 러너 경험 보고 (2026-09-02 → 2026-09-25) · 초안 v4

버전 v4

작성일 2026-09-26

문서 서로를 보완하는 에이전트로 하는 포트폴리오 규모의 자율 소프트웨어 유지보수

서로를 보완하는 에이전트로 하는 포트폴리오 규모의 자율 소프트웨어 유지보수

aidev 러너 경험 보고 (2026-09-02 → 2026-09-25)

hkjang · Claude(Fable 5.1)가 시스템과 이 보고서의 공동 저자 · 초안 v4, 2026-09-25 · English version: [English](#) · PDF: [한국어](#) · [English](#)

초록

한 사람이 소유한 46개의 중소 규모 저장소를 자율적으로 유지보수하는 러너 aidev 를 18일 동안 운영한 경험을 보고한다. 러너는 *회차*마다 저장소를 고정된 커밋에서 체크아웃하고, LLM 에이전트가 개선 하나를 고르고 구현하게 하고, 그 변경을 러너가 직접 검증하고, PR 을 열고, CI 와 독립 리뷰로 머지를 막고, 머지한 뒤 그 저장소의 관례대로 릴리즈한다. 기간 동안 러너는 1,031회차를 돌아 PR 443건을 열었고, 그중 244건을 사람이 키보드를 건드리지 않은 채 머지했으며, 릴리즈 331건을 게시했다. 모델 비용은 약 3,750달러였다.

설계의 핵심 질문은 "모델이 패치를 쓸 수 있는가"가 아니라 "지켜보는 사람 없이 쏟아지는 패치들을 어떻게 정직하게 유지하는가"였다. 답은 하나의 만능 에이전트에서 출발해 **자기 일을 스스로 승인할 수 없는 상보적 에이전트들**로 진화했다. 과제서를 쓰는 읽기 전용 *정찰*, 그것을 구현하는 *구현자*, diff 만 읽는 *비평가*, 비평가가 지목한 것만 고치는 *수리자*, 둘이 맞설 때 판정하는 *중재자*, *릴리즈 담당*, 멈춘 PR 을 매시간 되살리는 *PR 처리기*한 쌍, 그리고 거절과 회귀를 규칙으로 정제해 다른 에이전트들이 시작 전에 읽게 하는 *기록자*들이다. 에이전트들은 회차마다 한 장의 노트에 각자 무엇을 했고, 무엇이 확실하지 않은지를 적으며 협업한다.

이 글은 아키텍처, 무인 운영을 견딜 만하게 만든 안전 울타리(기준 커밋 고정, 러너 직접 검증, 비밀값·산출물 게이트, 보호 경로, 자동 강등이 있는 자율화 단계, SHA 고정 승인, 예산, 비상 정지), 표준 문서와 참조 구현으로 한 기능을 서른 저장소에 펼치는 *캠페인*장치, 그리고 지금까지의 측정치를 기술한다. 시스템이 매일 바뀌었기에 관찰된 전후 숫자로는 효과를 역할에 귀속시킬 수 없다. 그래서 판단 역할을 하나씩 빼는 **무작위 배경 비교 실험**(정찰, 회차 내 수리, 중재, 같은 계열 비평, 공유 노트, 정제된 교훈을 각각 뺀 arm 일곱)을 정의해 시작하고, 예비 연구 둘을 보고한다 — 교차 모델 비평 비교(PR 12건: 두 번째 계열이 모든 거절에 동의했고 승인 6건 중 2건을 추가로 거절)와 30일 머지 뒤 유지보수 측정(머지된 변경의 27%가 한 달 안에 사람이 고친 파일에 있음). 이어서 실제로 일어난 사고들과 거기서 나온 운영 노하우를 나열한다. 어렵게 배운 교훈 대부분은 프롬프트가 아니라 배관에 관한 것이었다. 루프 안에서 stdin 을 삼킨 자식 프로세스, 회차 도중에 죽은 스케줄러 프로세스, 캠페인 네 개를 한꺼번에 닫아 버린 탭 구분 필드, 침입처럼 보였던 알림 같은 것들이다.

1. 서론

저장소 마흔 개를 가진 개발자 한 사람은 매주 그 모두를 돌볼 수 없다. 의존성은 썩고, 가이드는 코드에서 멀어지고, 같은 기능(SSO, 메일 알림, 추적 스니펫, MCP 엔드포인트)을 서른 서비스에 같은 모양으로 넣어야 하며, 그 변경마다 테스트·리뷰·머지·릴리즈가 필요하다. 이 일감의 흥미로운 성질은 개별 변경이 어렵다는 데 있지 않다. 너무 많고, 대부분 정형이며, 잘못된 하나의 대가가 실체라는 데 있다. 깨진 릴리즈, 새어 나간 비밀값, 인증 우회.

aidev 는 이 일감을 LLM 에이전트에게 넘기되 *무엇을 무인으로 머지해도 되는가*에 대한 통제는 운영자가 쥐는 시도다. 2026-09-02 에 저장소마다 에이전트 하나를 돌리는 셸 스크립트로 시작해, 매일의 운영과 긴 사고 목록을 거쳐 열린 개의 이름 붙은 에이전트 역할, 모의 실행 하네스, 대시보드, 텔레그램 인터페이스, 그리고 몇 편의 표준 문서를 가진 시스템이 되었다.

이 보고의 기여는 넷이다.

1. 변경을 만드는 역할과 판단하는 역할이 서로 다른 세션·다른 도구이고 어떤 역할도 자기 산출물을 승인할 수 없는 **상보적 에이전트 아키텍처** (§3).
2. 모델의 판단이 아니라 값싼 기계적 게이트로 세운 **무인 머지의 안전 울타리** (§4).
3. **캠페인**: 표준 문서와 참조 구현으로 한 기능을 여러 저장소에 펼치고, 앞선 저장소의 실패에서 정제한 교훈을 뒤의 저장소에 주입하는 방법 (§5).
4. 판단 역할을 한 번에 하나씩 빼는 **무작위 배정 비교 실험**(arm 일곱, RQ1~RQ5, 2026-09-19 부터 진행 중)과 예비 연구 둘 — 교차 모델 비평 비교와 30일 머지 뒤 유지보수 측정 — 그리고 18일의 관찰 데이터가 뒷받침하는 것과 못하는 것 (§6, §7), 같은 것을 만드는 사람에게 건넬 **운영 노하우** (§8).

여기 기술한 모든 것은 저장소 `hkjang/aidev` 에 있다(러너 `bin/run.sh`, 프롬프트 `prompt.md · review-prompt.md · agents/*.md`, 문서 `AGENTS.md · README.md`). 숫자의 원본 데이터는 `docs/data/runs.jsonl` 과 `docs/data/usage.jsonl` 로 공개돼 있다.

2. 시스템

2.1 회차

회차가 일의 단위다. 러너(Windows 작업 스케줄러가 10분마다 깨우고 파일 잠금으로 직렬화되는 Bash 스크립트)는 저장소 하나를 고른 뒤:

1. **기준을 고정한다**. 기본 브랜치를 받아 SHA 를 기록하고 그 SHA 에서 새 git worktree 를 만든다. 이후의 모든 판단(diff, 보호 경로, 검증)은 이 고정된 기준에 대해 이뤄지고, 재검증 없이 기준이 움직였으면 머지를 거부한다.
2. **에이전트들을 돌린다** (§3). 격리된 홈 디렉터리, GitHub 자격 증명 없음, 프로세스 범위의 push 차단. 에이전트는 빌드하고 테스트를 돌릴 수 있지만 push 할 수 없고, 운영자의 비밀값을 읽을 수 없고, 러너의 정책 파일을 볼 수 없다.
3. **변경을 직접 검증한다**. 정책에 적힌 또는 자동 감지한 테스트·린트·빌드 명령(`go test ./...`, `npm test`, `pytest`, `gradlew test` 등)을 러너가 돌리고, diff 에서 비밀값을 찾고, 커밋된 빌드 산출물을 거부한다. 에이전트의 주장과 무관하다.
4. **PR 을 연다**. 에이전트의 원장 항목과, §3.5 이후로는 회차 노트를 싣는다.
5. **머지를 막는다**. 보호 경로(워크플로, 마이그레이션, 인증, 결제, 비밀값, LICENSE)를 건드리면 사람 몫으로 보류한다. 아니면 CI 가 두 번 연속 통과하고 독립 비평가가 승인해야 한다.
6. **머지하고 릴리즈한다**. 릴리즈 에이전트가 그 저장소의 이전 릴리즈들을 읽고 관례(태그, 버전 파일, 변경 이력, 워크플로, 첨부 자산)를 되풀이한다. 러너는 태그와 GitHub Release 가 있는지, 필수 자산이 이전 릴리즈에서 유도한 매니페스트와 맞는지 확인한다.
7. **기록한다**. 회차마다 구조화 기록(`runs.jsonl`: 결과, 단계별 상태와 사유, PR, SHA, 캠페인)과 증거 디렉터리(`state/runs/<id>/`)가 남고 대시보드가 다시 만들어진다.

2.2 결과 분류

회차는 여덟 결과 중 하나로 끝난다. `release-ready` (머지하고 검증된 릴리즈 게시), `merged` (머지, 릴리즈할 것 없음), `releasing` (머지, 릴리즈 미완), `review-pending` (PR 열림, 대기), `verify-failed` (러너 검증 또는 CI 실패), `no-change`, `error`, 그리고 나중에 더한 `infra-error` 와 `usage-limit` (저장소 잘못이 아닌 실패). 마지막 둘과 `error` 의 구분은 캠페인에서 중요해졌다 (§5.3).

2.3 회차 밖

회차 밖에서 도는 것들: 30분마다 죽은 스케줄러를 깨우고 잠금을 푸는 헬스체크, 머지 2~48시간 뒤 main CI 와 되돌림을 확인하는 회귀 감시, 오류로 끝난 회차를 다시 큐에 넣는 *fixer* 트랙, 매시간 도는 *PR 처리기* (§3.4), 기록자들 (§3.6), 그리고 운영자가 폰에서 "weekly 왜 멈췄어" 라고 묻거나 "moina PR 승인해" 라고 말할 수 있는 텔레그램 코파 일렛.

3. 상보적 에이전트

3.1 왜 하나로는 모자랐나

처음 두 주는 "개선" 에이전트 하나가 전부 했다. 저장소를 읽고, 아이디어를 나열하고, 하나를 고르고, 구현하고, 테스트를 돌리고, 커밋하고, 원장을 썼다. 첫날부터 머지된 릴리즈를 만들어 냈으니 동작하긴 했다. 그러나 실패 양상이 일관됐다.

- 가치 있는 것이 아니라 끝내기 쉬운 것을 골랐다. 같은 세션에서 구현까지 해야 하는 에이전트는 자신이 끝낼 수 있다고 확신하는 아이디어로 기운다.
- 자기 숙제를 자기가 채점했다. 원장에는 "검증함" 이라 적혔고 실제로 검증을 돌린 경우도 많았지만, 커밋이 나온 회차의 11% 는 러너의 독립 검증에 실패했고, 거기까지 간 diff 의 18% 는 독립 비평가가 거절했다.
- 매번 저장소를 새로 배웠다. 45분 세션의 앞 10분쯤은 README, 구조, 테스트 설정을 읽는 데 썼다. 하루 마흔 번 되풀이되는 같은 일이었다.
- 틀렸을 때 아무도 반박하지 않았다. 거절된 리뷰는 코멘트와 함께 PR 을 열어 둘 뿐이었다. 그 코멘트를 읽는 것은 없었다.

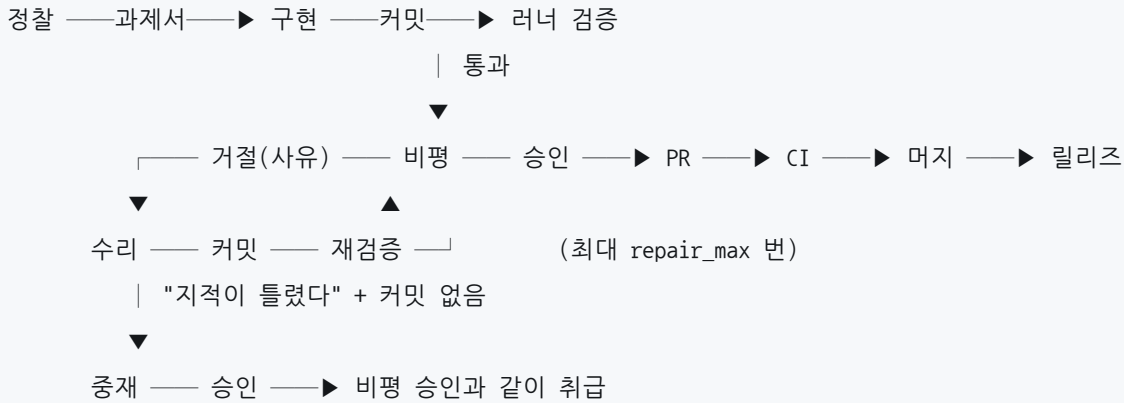
상보적 역할로의 개편(2026-09-18)은 이 관찰과 한 가지 원칙에서 나왔다. *변경을 만드는 역할과 판단하는 역할은 다른 세션이어야 하고, 판단하는 쪽은 편집할 수 없어야 한다.*

3.2 명부

역할	세션 도구	만드는 것	누가 검사하나
정찰	읽기 전용, 테스트 실행 가능, 커밋 불가(남긴 변경은 러너가 버림)	brief.md — 과제, 수용 기준, 건드릴 파일, 실제 검증 명령, 위험, 차선 후보; 갱신된 아이디어 백로그; 프로젝트 프로필	구현자가 과제를 코드로 확인하고 채택 / 차선 / 기각 을 기록
구현	편집·실행·커밋	커밋, 원장 항목, 아이디어	러너 검증, 그다음 비평가
비평	읽기·실행, 편집 불가	review.json — 판정, 파일·줄이 있는 사유, 위험도	형식 게이트; approve 만 머지 경로로
수리	편집·커밋(비평이 지목한 것만)	수리 커밋, fix-summary.md	재검증, 그다음 비평가 재심; 실패한 수리는 되돌림
중재	읽기·실행, 편집 불가	arbiter.json — 사유별 "확인됨 / 반증됨"	수리가 손대지 않고 반론만 남겼을 때만
릴리즈	편집, 태그, 자산 빌드	release.json , 태그, 자산	릴리즈 게이트, 자산 매니페스트, 워크플로 결과
PR 수리 / PR 심사	수리·비평과 같음, PR 브랜치 위에서 매시간	수리 커밋; recommend 가 있는 review.json	러너 검증; 승인 스위치 CI 뒤 승인 SHA 만 머지
기록자	없음(오프라인 정제)	campaign-lessons/<id>.md , operator-preferences.md	입력 해시 캐시; 사람이 대시보드에서 읽음
설계자	읽기 전용 + drafts/ 쓰기	캠페인 초안: 표준, 대상, 예산	사람이 시작
코파일럿	스크립트 하나의 동사만	운영 조치	운영자(대화)

정찰과 비평은 코드를 바꿀 수 없는 두 역할이다. 그 비대칭이 요점이다. 구현할 수 없는 정찰은 가치로 과제를 고르고, 편집할 수 없는 비평은 "고치고 승인" 할 수 없다.

3.3 회차 안의 루프



비평은 PR을 열기 전에 돌지만, 그 변경이 자동 머지될 때만(자율화 low-risk 이상, 보호 경로 없음) 돈다. 보호 경로를 건드린 변경은 어차피 사람이나 PR 심사가 보므로 회차 안의 비평을 건너뛴다. 거기서 비평을 돌리면 아무 결정도 더해지지 않은 채 비용만 두 배가 된다. 루프가 승인 없이 끝나면 PR을 열고 비평 사유와 함께 보류하며, 한 시간 뒤 PR 처리기가 새 세션에서 다시 잡는다. 한 모델의 고집이 이어지지 않도록 일부러 다른 세션이다.

3.4 PR이 열린 뒤: PR 처리기

2026-09-17 기준 러너가 열었지만 아무도 보지 않은 PR이 191건이었다. 보호 경로 55, 비평 거절 52, CI 실패 25, CI 없음 16, 충돌 15. 운영자는 "별도 프로세스로 한 시간마다 문제를 파악해 머지·릴리즈되게 하는 체계"를 요구했다.

처리기는 멈춘 PR마다 회차의 단계 기록과 GitHub의 현재 상태로 원인을 진단하고 조치한다.

- 충돌 → worktree에서 리베이스, 재검증, `--force-with-lease` 푸시;
- CI 실패·리뷰 거절 → PR 브랜치 위에서 실패 로그나 거절 사유를 든 수리 세션, 러너 검증, 푸시;
- 보호 경로·자율화 보류 → 사람을 대신하는, 따라서 불확실하면 거절하는 더 엄격한 심사 세션(`shepherd-review-prompt.md`)으로 바로.

심사자는 `recommend` 필드(`merge`, `fix`, `human`)를 돌려주고 러너는 그것을 그대로 따른다. 운영자의 지시가 "승인하는 쪽의 추천대로 하게 하라"였고, 그것을 뒤집던 러너 쪽 위험도 임계값을 없었다. `merge` 권고는 정확한 SHA와 함께 `approvals.jsonl`에 `by: shepherd`로 기록되고, 평소의 승인 스왑이 CI 뒤 그 SHA만 머지한다. 처리기 자체는 `gh pr merge`를 부르지 않는다.

처리기가 구조적으로 손대지 않는 것: 워크플로·비밀값·결제·LICENSE를 바꾼 PR, 자율화가 `approve` 이하인 프로젝트, CI가 아예 없는 커밋, 이미 두 번 실패한 것. 이것들은 진단을 달아 작업함에 "사람 필요"로 올라간다. 첫 이틀 동안 142건의 조치를 했다. 거절 44, 수정 푸시 28, 승인 25, 사람에게 36 (§6.4).

3.5 역할들이 어떻게 말하나: 회차 노트

산출물(`brief.md`, `review.json`, `fix-summary.md`)은 역할 사이의 계약이다. 협업에는 모자랐다. 비평은 구현이 무엇을 확신하지 못했는지 몰랐고, 릴리즈는 비평이 승인하면서도 무엇을 걱정했는지 몰랐으며, 처리기는 매 PR을 아무것도 모른 채 시작했다.

2026-09-19 부터 모든 회차에 `journal.md` 한 장이 있다. 각 역할이 짧은 절을 덧붙인다. 정찰은 왜 이 과제를 골랐고 무엇을 추측했는지, 구현은 무엇을 바꿨고 **무엇이 확실하지 않으며 무엇을 검증하지 못했는지**, 일부러 하지 않은 것은 무엇인지, 비평은 무엇을 확인했고 무엇을 못 봤으며 승인해도 남는 우려가 무엇인지, 수리는 어떤 지적이 맞았고 틀렸는지, 중재는 누가 왜 맞았는지. 러너의 단계 판정도 같은 파일에 시간순으로 끼어든다. 다음 역할은 노트를 먼저 읽으라는 지시를 받고, 특히 비평은 구현의 "확신 없는 곳" 부터 시작하라고, 그 목록이 비어 있으면 "전부 확신한다는 뜻이니 그 확신을 시험하라" 고 지시받는다.

노트는 옮겨 다닌다. PR 본문에 접혀 들어가고, 릴리즈 에이전트가 받는 변경 요약의 일부가 되며, 처리기는 PR 을 연 회차의 노트를 이어받는다. 회차를 넘어 루프를 닫는 되먹임이 둘 더 있다. 과제서에 대한 구현자의 판정(채택 / 차선 / 기각)은 다음 정찰에게 "최근 과제서 여섯 건이 어떻게 됐나" 로 보여지고, 중재가 비평을 뒤집은 사례는 다음 비평에게 "이 저장소에서 지나치게 엄격했던 자리" 로 보여진다.

3.6 저장소를 가로지르는 학습: 기록자

오프라인 정제기 둘이 증거를 규칙으로 바꾸어 일하는 에이전트들이 시작 전에 읽게 한다.

캠페인 교훈. 캠페인마다 기록자가 그 캠페인 모든 저장소의 비평 거절 사유, 처리기 거절, 머지 뒤 회귀를 모아 해시하고, 해시가 바뀌었을 때만 모델에게 최대 여덟 개의 *일반* 규칙("X 전에 소스에서 Y 를 확인한다" 이지 "저장소 Z 40행" 이 아님)으로 정제하게 한다. 규칙은 이후 모든 회차의 캠페인 노트와 PR 심사의 보류 사유에 주입된다. 첫 정제(2026-09-17)는 증거 36건에서 규칙 25개를 냈다. 그중 하나 — "소문자 사본의 바이트 오프셋을 원본 문자열에 적용하지 말 것, U+0130·U+212A 로 테스트할 것" — 는 규칙이 생기기 전에 두 저장소에서 각각 비싸게 발견된 것이었다.

운영자 취향. 같은 장치를 운영자 자신의 말에 적용했다. 사람이 반려하며 남긴 이유(이제 러너가 운영자의 마지막 PR 코멘트를 교훈에 심는다), 비상 정지 사유, 코파일럿에 한 지시. 증거 19건에서 규칙 10개가 나왔다. "손으로 주입한 대역이 아니라 프로덕션 배선을 통과하는 테스트로 증명할 것", "관찰 가능한 동작이 바뀌지 않는 변경은 반려 사유다" 같은 것들이다. 이것이 구현자와 심사자 프롬프트 앞에 붙어, 운영자는 같은 말을 두 번 하지 않아도 된다.

3.7 문헌이 바꾼 것 (2026-09-19)

§9 의 관련 연구를 읽은 뒤 장치 셋을 더했고 §6.6~6.8 에서 쟀다. 판정의 자기 선호 [10, 11] 에 맞서 비평을 다른 모델 계열(Codex)로 보내는 프로젝트별 `critic_engine` 스위치, 에이전트 코드가 머지율이 시사하는 것보다 더 많은 머지 뒤 유지보수를 안고 있다는 발견 [24, 25] 에서 나온 30일 머지 뒤 수정 유지보수 측정, 그리고 성적표의 MAST 기반 실패 관점 [16]. 앞의 둘은 adev 자신의 숫자를 공개된 현장 연구와 비교할 수 있는 첫 자리이기도 하다.

4. 안전 울타리

무인 머지는 값싼 기계적 검사가 충분히 강해서 비싸고 틀릴 수 있는 검사(모델 판단, 사람의 주의)가 마지막 방어선이 아닐 때만 받아들일 만하다. adev 의 울타리를 적용 순서대로:

- **기준 고정과 새 worktree.** 움직이는 대상에 대해 판단하지 않는다. 머지 전에 기준이 움직였으면 리베이스·재검증하거나 보류한다.
- **러너 직접 검증.** 테스트·린트·빌드는 에이전트가 보고하는 것이 아니라 러너가 돌린다. 1MB 넘는 바이너리(문서 PDF 는 15MB)는 산출물 검사가 거부한다. 한 회차가 20MB 실행 파일을 커밋한 적이 있다.
- **비밀값 게이트** — diff, 원장, 이제는 노트에도. 자리표시자와 로컬 DSN 은 일주일의 오탐 뒤 허용 목록에 넣었다.

- **보호 경로**(`state/default.guard`): 워크플로, 마이그레이션, `auth/session/oauth`, 결제, 비밀값, LICENSE. 건드리면 사람 몫으로 — 처리기 이후로는 명시적 금지 목록(워크플로·비밀값·결제·LICENSE 는 사람만)이 있는 더 엄격한 심사자 몫으로 보류한다.
- **독립 리뷰**와 형식 게이트: 형식이 깨졌거나 없는 판정은 승인이 아니라 보류다.
- **CI 게이트**: 정확한 SHA 의 `check-run` 을 받아 두 번 연속 `success` 여야 하고(잡이 늦게 등록된다), CI 가 없는 저장소는 정책이 허용하지 않으면 막힌다.
- **프로젝트별 자율화 단계** — `analyze < pr < approve < low-risk < release` — 올리는 것은 사람이 파일을 고쳐야만 하고, 롤백이나 회귀가 감지되면 러너가 **자동으로 내린다**. 기간 중 두 프로젝트가 강등됐다.
- **SHA 고정 승인**. 사람(또는 처리기)은 특정 커밋을 승인한다. 브랜치가 바뀌면 라벨을 떼고 다시 받아야 한다.
- **세 층의 예산**: 단계별, 회차별(합이 그날 남은 상한에 들어야 시작), 일별. 캠페인과 처리기는 자체 예산을 가져, 탐색 작업의 상한이 마무리 작업을 막지 않게 한다.
- **비상 정지**: `STOP`, `STOP-merge`, `STOP-release`, `STOP-〈프로젝트〉`, `NO-SHEPHERD`, `NO-COPILOT`. 모든 단계 경계에서 확인한다.
- **증거**. 회차마다 단계, 게이트 판정, 리뷰 JSON, CI 스냅샷, 노트, 출처 기록(정책 버전, 프롬프트 해시, 러너 버전)이 남는다.

이 중 어느 것도 모델이 옳을 필요가 없다. 그것이 설계 목표다. 판단이 필요한 곳에 모델을 쓰고, 울타리가 잘못된 판단을 값싸게 만든다.

5. 캠페인

5.1 문제

운영자의 저장소들은 스택(Go 백엔드, React SPA, Keycloak SSO)을 공유하고 같은 능력을 되풀이해 필요로 한다. 조용한 SSO, CSP 가 있는 추적 스니펫, SMTP 메일 알림, 서비스 간 문서 전달, 화면 캡처가 있는 완전한 가이드, MCP 엔드포인트의 OAuth — 각각이 스무에서 서른 저장소에 같은 변경을 뜻했다.

5.2 장치

캠페인은 `campaigns.json` 의 항목이다. id, 구현자에게 주는 지시문 형태의 목표, 대상 저장소 목록, 예산, 기한, 그리고 이 일이 건드릴 것으로 *예상되는* 보호 경로 조각(보호는 그대로 PR 을 잡되 대상마다 경보를 울리지는 않게). 목표는 aidev 저장소의 **표준 문서**와, 그 능력이 이미 잘 구현된 저장소인 **참조 구현**을 가리키며, "코드를 베끼지 말고 이 저장소의 언어와 구조에 맞게 같은 능력을 만들라" 고 명시한다.

러너는 캠페인을 유한한 일감으로 다룬다. 대상마다 한 회차, PR 까지 간 대상은 완료, 오류로 끝난 대상은 최대 세 번 재 시도 뒤 막힘으로 보고, 대상이 남지 않거나 예산·기한이 다하면 종료. 캠페인은 탐색 회차보다 우선하고 서로 돌아가며 차례를 받아, 나중 캠페인이 앞선 캠페인에 굶지 않는다.

5.3 무슨 일이 있었나

기간 중 캠페인 여섯이 총 225회차를 돌았다.

캠페인	대상	처리	완료(머지)	PR 대기	막힘	비용
guides (실제 화면 캡처가 있는 사용자·관리자 가이드)	25	22 (88%)	6	16	2	\$455
silent-ssso (OIDC prompt=none)	33	31 (93%)	5	25	0	\$231
tracking (관리자가 붙이는 스탬프와 CSP nonce)	33	29 (87%)	11	18	1	\$360
mail (SMTP 릴레이 알림)	33	29 (87%)	2	27	1	\$356
handoff (서비스 간 문서 전달)	15	12 (80%)	1	11	0	\$128
mcp-oauth (/mcp 의 Keycloak 토큰)	31	28 (90%)	8	20	0	\$373

두 가지가 두드러진다. 첫째, 캠페인이 PR 에 도달한 비율은 높았고(며칠 만에 대상의 8093%) 대상당 비용은 낮았다 (\$20달러). 둘째, 그 PR 이 머지된 비율은 낮았다. 인증이나 마이그레이션 기능이란 원래 그런 것이라 거의 전부 보호 경로를 건드렸고, 사람을 기다렸기 때문이다. 이것이 처리기를 만든 이유이자, 운영자가 이 경로에 대해 더 엄격한 심사를 조건으로 AI 의 AI PR 승인을 받아들인 이유다.

캠페인 고유의 사고 셋이 장치를 다듬었다. 여러 줄로 쓴 목표가 탭 구분 파싱을 깨뜨려 until 이 빈 값으로 읽혀 캠페인 이 시작하자마자 기한 만료로 스스로 닫혔다(목표를 base64 로 씌는 것으로 수정). 같은 파싱의 열 하나가 어긋나 캠페인 네 개가 "대상 전부 완료" 로 한꺼번에 닫혔다(빈 대상 목록으로는 닫지 않도록 수정). 디렉터리는 사라졌는데 locked 등록만 남은 worktree 가 한 저장소를 다섯 번 연속 실패시켜 캠페인에서 제외시켰다(infra-error 를 error 와 구분해 인프라 실패는 스트라이크로 세지 않도록 수정).

6. 실험 설계와 측정

6.0 무작위 배정 비교 실험 (RQ1~RQ5)

이 절 앞의 모든 것은 거의 매일 바뀐 시스템을 기술한다. 그 변경들 전후의 비교는 처치 비교가 아니라 시점 비교다. 그래서 2026-09-19 부터 러너는 무작위 배정 실험을 돌린다. 모든 개선 회차가 run id 해시로 일곱 arm 중 하나에 배정되고, arm 은 그 회차에 한해 프로젝트 정책을 덮어쓰며, arm 은 회차와 함께 기록된다. arm 은 판단 역할을 한 번에 하나씩 뺀다. 어느 arm 도 기계적 안전 게이트(검증, 보호 경로, CI, SHA 고정 승인)는 빠지 않는다.

arm	덧어쓰기	답하는 질문
baseline (가중치 2)	없음: 정찰, 비평, 수리 1회, 중재, 노트, 교환, Claude 비평	기준
no-scout	구현자가 과제를 직접 고름	RQ1 읽기 전용 정찰이 과제를 정하면 검증 통과·PR 도달·과제서 채택이 나아지는가?
no-repair	비평 거절이면 PR 을 열고 보류, 회차 내 수리 없음	RQ2 회차 내 수리와 중재가 검토 대기·머지율·비용을 어떻게 바꾸나?
no-arbiter	수리는 하되 수리·비평이 맞설 때 제3 세션 없음	RQ2
codex-critic	비평을 OpenAI Codex 세션이 함	RQ3 다른 계열의 비평가는 더 많이 거절하는가, 그 승인은 머지 뒤 더 나은가?
no-journal	역할들이 앞선 역할의 노트를 보지 않음	RQ4 구현의 "확신 없음" 노트를 읽는 것이 비평 거절과 수리 성공을 바꾸나?
no-lessons	캠페인 교환·운영자 취향 주입 없음	RQ5 정제된 되먹임이 같은 유형의 거절 재발을 줄이나?

배정은 저장소와 캠페인으로 암묵적으로 총화되고(둘 다 기록됨), 분석(`bin/exp-analyze.py`)은 arm 별로 검증 통과율, 비평 거절률, PR 도달, 머지율, 검토 대기율, 회차당·머지당 비용, 과제서 채택, 수리 성공, 중재 결과, §6.8 의 30일 머지 뒤 사람 수정률을 부트스트랩 95% 신뢰구간과 baseline 대비 차이로 보고한다. 하루 약 60회차면 2주에 arm 당 100~200회차가 된다. RQ3·RQ5 의 결과 지표는 머지 30일 뒤에 무르익으므로 1차 판독은 월말, 2차는 10월 중순으로 계획한다. 아래 예비 연구 E1·E2 는 실험 시작 전에 돌린 것이며 그렇게 보고한다. 실험 정의는 `state/experiment.json` , 결과는 `docs/paper/experiments/ab-roles-2026-09.md` 에 공개한다.

6.1 규모 (관찰, 2026-09-02 → 09-19)

아래 숫자는 2026-09-19 기준(18일) `runs.jsonl` 과 `usage.jsonl` 에서 나왔다. 처음 사흘은 구조화 결과 필드가 생기기 전이라 초기 249회차는 결과 문장으로 분류했다. 운영된 그대로의 시스템을 기술할 뿐 처치를 비교하지 않는다.

건드린 저장소	46
회차(전체)	1,031
개선 회차	704
연 PR (고유)	443
러너가 수행한 머지 (단계 merge done)	244
스웽에 들어간 사람 승인 / 처리기 승인	60 / 25
게시한 릴리즈 (고유)	331
하루 회차 (중앙값 / 최대)	58 / 102

6.2 품질 게이트

게이트	통과	실패 / 보류	실패율
러너 검증 (커밋이 있는 회차)	447	57	11%
독립 비평	273	59	18%
CI (정확한 SHA, 두 번 연속)	228	실패 26, CI 없음 17, 시간 초과 3	17%
보호 경로	—	107 보류 (캠페인이 예상한 50 포함)	—
머지 시점 충돌	—	17 실패, 15 리베이스	—

비평 거절에는 298개의 개별 사유가 있었고 자체 평가 위험도는 medium 87, low 12, high 4 였다. 승인되어 머지된 273건 중 지금까지 되돌리거나 롤백된 것은 **없다**(기간 중 롤백 1건과 강등 2건은 리뷰된 코드가 아니라 릴리즈 워크플로 실패에서 왔다 — 다만 관찰 창에 대해서는 §7 을 보라).

6.3 크기와 비용

변경 중앙값	파일 7개, +330줄
테스트 파일이 포함된 변경	93%
모델 비용 합계	1,343세션에 3,746달러
단계별	구현 \$2,825 · 비평 \$522 · 릴리즈 \$378 · 정찰 \$16 (이틀) · 수리 \$3
머지된 변경당 (전체 비용 ÷ 머지)	≈ 8달러
세션 길이 중앙값	구현 9.2분 (p90 21) · 비평 3.2분 · 릴리즈 3.0분 · 정찰 3.7분
출력 토큰 / 캐시 읽기	2,900만 / 39억

비용은 모델 자체 추정치이고 운영자는 정액제라, 청구액이 아니라 *사용량*을 측정한 것이다. 캐시 읽기 수치는 세션별 문맥(프로필, 노트, 교훈)을 더하는 것이 왜 싼지를 보여 준다. 프롬프트의 거의 전부가 캐시 적중이다.

6.4 PR 처리기, 1~2일차

조치	건수
진단: 비평 거절 / 보호 경로 / 충돌 / CI 실패 / CI 없음	59 / 44 / 22 / 9 / 3
심사 거절 (다음 시간에 수리)	44
수리가 검증된 수정을 푸시	28
수리 실패	5
리베이스	3
심사 승인 → 승인 스왑	25
진단을 달아 사람에게	36

일주일 동안 움직이지 않던 적체에 대해 이틀에 승인 25건, 비용 약 105달러. 기간 중 가장 눈에 띄는 생산성 향상이다. 사람에게 넘긴 36건은 거의 모두 의도된 종류다. 워크플로 파일, approve 단계 프로젝트, 두 번 실패한 PR.

6.5 상보 루프, 1일차

정찰/비평/수리 루프는 이 글을 쓰는 시점에 하루 돌았다. 정찰 세션 11회(과제서 9건, 실패 2건), 회차 안 수리 2회(모두 성공). 루프가 거절률에 미치는 효과를 말하기엔 너무 적다. 장치가 자리를 잡았고 대시보드의 성적표가 쌓아 갈 것이라는 뜻에서 적어 둔다.

6.6 예비 연구 E1 — 교차 모델 비평

질문. audev 의 구현자와 비평가는 같은 모델 계열이다. LLM 심판은 익숙한 텍스트를 선호한다 [10, 11]. 다른 계열의 비평가는 우리 비평가와 어긋나는가, 어긋난다면 어느 방향으로인가?

방법. Claude 비평가가 승인하고 머지된 최근 PR 6건과 거절한 최근 6건(2026-09-12 → 09-19, 저장소 12개)을 골랐다. 각각에 대해 OpenAI Codex 세션(codex-cli 0.154, 읽기 전용 샌드박스, 노트 없음)이 PR head 에서 고정된 기준에 대해 동일한 review-prompt.md 를 받고 같은 JSON 을 냈다. 스크립트 bin/exp-cross-critic.sh , 원본 docs/paper/experiments/cross-critic.jsonl .

결과.

Claude 비평	n	Codex 승인	Codex 거절
승인(머지됨)	6	4	2
거절	6	0	6

판정 일치는 12건 중 10건. 거절 6건은 Codex 도 모두 거절했고, 그중 3건(seaton 의 빈 범위 우회, sqlon 의 삭제된 설정 정의, AgentHub 의 검증 없는 DLP class 복사)은 같은 줄의 같은 결함을 지목했으며 나머지 3건은 다른 구체적인 이유로 거절했다. 승인 6건 중 2건을 Codex 는 Claude 비평이 제기하지 않은, 확인 가능한 구체적 사유로 거절했다. aiportal-py 에서는 피드백 태스크가 취소되고 DB 풀이 닫힌 뒤에도 executor 가 계속 돌아 늦게 돌아온 배치가 결과를 저장하지 못하는 종료 순서, aiportal-front 에서는 새 직접 의존성이 필요함을 "증명" 하지만 lockfile 에 이미 있는 전이 의존성으로도 통과하는 테스트. 둘 다 그럴듯한 실제 결함이고, 아직 관찰된 회귀를 일으키지는 않았다. Codex 세션은 중앙값 72초, 입력 토큰 약 20만.

해석. 이 작은 표본에서 두 계열은 명백히 깨진 것에는 일치하고, 두 번째 계열은 첫 번째가 통과시킨 것에 대해 더 엄격했다. 승인 쪽의 자기 선호 효과와 일관된다. 이는 우리가 취한 두 결정을 뒷받침한다. 머지된 코드에 출처가 다른 두 번째 시선을 두는 것(PR 심사자와 아래의 머지 뒤 측정), 그리고 프로젝트가 비평을 다른 계열로 보낼 수 있는 critic_engine 옵션. 두 번째 계열의 오거절률은 알려 주지 않는다. 그것은 그 두 건에 대한 중재자의 판정이 필요하며, 큐에 넣어 두었다.

6.7 실패 분류 (MAST 관점)

MAST 의 세 범주 [16] 를 14일 창(실패 사건 $n = 299$, 한 회차가 둘 이상에 들 수 있음)에 적용하면:

범주	건수	audev 에서 무엇인가
시스템 설계·인프라	154	예산 보류, 사용량 한도, 깃 worktree, 스케줄러 종료
에이전트 간 불일치	59	비평 거절, 구현자가 기각·차선 처리한 과제서, 아무것도 바꾸지 못한 수리
검증·종료	86	러너 검증 실패, CI 실패·시간 초과, 중재 거절

우리 실패의 절반은 에이전트가 아니라 러너 자신의 배관이다. §8.1 과, 다중 에이전트 실패의 많은 부분이 모델 오류가 아니라 명세·시스템 설계 문제라는 Cemri 외의 발견과 일관된다. 이 분류는 이제 대시보드 성적표의 고정 행이다.

6.8 예비 연구 E2 — 머지 뒤 수정 유지보수

질문. Xia 와 Miller [24] 는 에이전트 코드가 머지 뒤 더 많은 수정 유지보수를 필요로 함을 발견했다. 우리 회귀 감시는 2~48시간만 본다. adev 가 머지한 변경에 30일 창은 무엇을 보여 주는가?

방법. 러너가 머지한 PR 마다($n = 244$, 7일 이상 관찰 166) `bin/postmerge.py` 가 그 PR 이 건드린 파일을 잡고, 30일 안에 기본 브랜치에서 그 파일 중 하나라도 건드린, 제목이 수정으로 읽히는 커밋(fix, hotfix, revert, bug, regress 또는 한국어 상당어)을 센다. 러너의 뒤 회차는 "버그 수정" 과제로 기울므로, 커밋을 뒤 러너 PR 이 만든 것과 사람이 만든 것으로 나눴다.

결과.

	해당 커밋이 있는 PR	비율
30일 안 같은 파일에 수정 제목 커밋	118 / 166	71%
... 그중 사람이 만든 것(뒤 러너 회차 아님)	45 / 166	27%
승인 주체별: 자동 머지 / 사람 승인	40/155 / 5/11	26% / 45%
비평 위험도별: low / medium	36/137 / 4/17	26% / 24%
첫 수정 제목 커밋까지 중앙값	4일	

해석. 이것은 "머지된 변경이 고쳐져야 했다" 의 상한이다. 같은 파일을 건드린 뒤의 수정이 반드시 그 변경의 수정은 아니고, 넓은 사람 수정 하나(Vendra 의 권한 조이기)가 PR 여섯 건에 한꺼번에 세어졌다. 그럼에도 머지된 변경의 4분의 1은 한 달 안에 사람이 고친 파일에 있고, 비평의 위험도 표시는 그것을 가르지 못했다(26% 대 24%). 그래서 §6.2 의 회귀 0 은 그것이 무엇인지 그대로 — 48시간짜리 진술 — 로 다루고, 30일 숫자를 단서와 함께 대시보드에 공개한다. 후속 둘을 큐에 넣었다. 파일이 아니라 줄(blame)로 수정을 변경에 귀속시키는 것, 그리고 자율화를 올리기 전에 프로젝트 별 30일 비율로 관찰 창을 정하는 것.

6.9 재집계 (2026-09-25) — 두 번의 데이터 보정 뒤

위 6.1~6.8 은 2026-09-19 시점의 집계다. 그 뒤 기록 자체에서 두 가지 결함을 찾아 고쳤고(§7 참조), 보정한 뒤의 숫자는 다르다. 두 결함은 모두 적게 세는 쪽이었다.

1. **사라진 승인 회차** — 승인 스왑이 머지 뒤 릴리즈까지 간 회차만 골라 기록에서 빠뜨렸다(71건). 56건을 머지 커밋 시각으로 되살렸다.
2. **초기 형식 회차** — 첫 나흘(9-02~05)의 249 회차에는 구조화된 `outcome` 이 없어 이후의 모든 집계·분석에서 빠져 있었다. 결과 문장으로 246건의 결말을 복원했다. 하필 관찰 기간이 가장 긴 표본이라 머지 뒤 분석에 가장 크게 기여한다.

	2026-09-19 (18일)	2026-09-25 (24일, 보정 후)
건드린 저장소	46	48
회차(전체)	1,031	1,441
개선 회차	704	1,045
연 PR (고유)	443	870
러너가 수행한 머지 (단계 merge done)	244	465
스왑에 들어간 사람 승인 / 처리기 승인	60 / 25	60 / 65
게시한 릴리즈 (고유)	331	464
하루 회차 (중앙값 / 최대)	58 / 102	61 / 103
누적 비용	—	\$5,050 (회차당 \$3.50)

머지 뒤 수정 필요율(E2)의 표본도 함께 커졌다 — 머지된 PR 459건(9-19 판)에서 655건으로, 관찰 7일 이상은 269건에서 **459건**으로. 표본이 커지자 승인 주체별 비교의 방향이 뒤집혔다.

승인 주체	표본(관찰 7일 ↑)	30일 안에 수정 커밋	그중 사람이 고친 것
러너 자동/심사 승인	423	70.9%	24.1%
사람 승인	48	70.8%	18.8%

작은 표본(각각 221·13건)에서는 AI 승인 쪽이 사람 승인보다 사람 손을 덜 타는 것처럼 보였다. 표본을 키우니 반대가 됐다. 두 수치 모두 무작위 배정이 아니다 — 사람은 보호 경로를 건드리는 PR 만 본다(§4), 즉 애초에 더 어려운 것을 본다. 그래도 방향이 표본 크기에 따라 뒤집혔다는 사실 자체가, 이 축에서 지금 결론을 내리면 안 된다는 뜻이다. 30일 창이 실제로 닫히는 것은 10월 초다 (현재 관찰 20일 이상 192건, 30일 이상 0건).

변경 규모가 뒤탈을 예측한다 (2026-09-26). 관찰 7일 이상인 머지 PR 471건을 건드린 파일 수로 나누면 30일 안에 사람이 같은 파일을 다시 고친 비율이 단조 증가한다.

프로덕션 파일 수	표본	30일 안에 사람이 다시 고침
1~3	130	14%
4~9	236	25%
10 이상	105	33%

무작위 배정이 아니므로 인과로 읽을 수는 없다 — 큰 변경은 애초에 더 어려운 문제일 수 있다. 그래도 2.4배의 차이는 운영 지침을 바꿀 만했다. 경찰과 구현 프롬프트에 "프로덕션 코드 파일 여섯 개 안쪽"을 근거(이 표)와 함께 넣었고, 성적표가 규모별 비율을 계속 센다. 다음 판에서 지침 전후를 비교한다.

저장소별 편차도 크다 — 같은 러너·같은 게이트인데 pii-masker·ptium·jikim 은 사람 수정 0%이고 Quantoss 는 80%(10건), Vendra 55%(20건)이다. 성적표가 상위 다섯을 띄우고, 70% 를 넘으면 자율화 단계를 낮추거나 검증을 늘리라고 권한다.

이 숫자들은 2026-09-26 에 한 번 크게 바뀌었다. 그때까지 릴리즈 커밋이 "사람이 고친 것" 으로 집계되고 있었다 — `release_project` 는 PR 없이 기본 브랜치에 바로 커밋하고 태그를 달아서 PR 기반으로 모으던 "러너가 만든 커밋" 집합에 들어가지 않았고, 하필 이 저장소들의 릴리즈 커밋 제목이 `fix: release the ... for v0.2.22` 처럼 `fix` 로 시작하며 버전 파일과 CHANGELOG 를 건드린다. 그래서 릴리즈를 할 때마다 그 저장소의 과거 PR 전부가 "사람이 다시 고쳤다" 로 한 건씩 늘었다. 자주 릴리즈하는 저장소일수록 나빠 보였다 — jikim 은 13건 전부 100% 였는데 고치고 나니 0% 다. 태그가 가리키는 커밋을 러너 커밋으로 넣어 바로잡았고, 전체 사람 수정률은 31%에서 24%로 내려갔다.

6.10 중간 판독 (2026-09-26, 실험 8일차)

실험은 10-03 까지만, 마감 때문에 이 시점의 판독을 싣는다. 엔진 혼입 (§7) 을 걷어내고 클로드가 구현한 회차만 놓고 arm 별 머지율을 baseline 과 비교했다(2표본 비율 z 검정).

arm	회차(클로드)	머지율	baseline 대비	p
baseline	50	64%	—	—
no-scout	20	95%	+31%p	0.008
no-repair	24	71%	+7%p	0.56
no-arbiter	27	70%	+6%p	0.57
no-journal	33	70%	+6%p	0.59
no-lessons	22	68%	+4%p	0.73
codex-critic	31	23%	-41%p	0.0003

여섯 번 비교했으므로 Bonferroni 기준은 0.0083 이다 — codex-critic 은 확실히 넘고, no-scout 은 경계에 걸쳐 있다. 읽을 수 있는 것은 둘이다.

RQ1(역할 분리). 정찰을 빼면 머지율이 오히려 올라갔다(64% → 95%). 정찰은 회차당 약 \$0.7~2 를 쓰고 "무엇을 할지" 를 정해 주는데, 그 과제가 구현자가 스스로 고른 것보다 덜 머지된다. 해석은 조심해야 한다 — 정찰이 더 어려운 과제를 고르는 것일 수도 있다(정찰 회차의 과제서 채택률은 95% 이므로 구현자가 거부해서가 아니다). 그래도 "읽기 전용 세션이 방향을 정하면 낫다" 는 이 시스템의 전제는 머지율로는 지지되지 않는다.

RQ3(심판의 계열). 다른 계열(Codex)이 비평하면 머지율이 23% 로 떨어진다. 거절이 많아서가 아니라 검토 대기로 남는 비율이 크게 늘어서다 (§6.4). 자기 선호 편향을 피하려고 계열을 바꾸면 그 대가로 처리량을 잃는다. 그 거절들이 진짜 결함이었는지는 머지 뒤 30일 지표로만 답할 수 있는데, 지금 표본으로는 arm 별 차이가 안 보인다.

나머지 넷(수리·중재·노트·교훈)은 머지율에서 baseline 과 구분되지 않는다. 없다는 뜻이 아니라 이 표본(arm 당 20~33회차)으로는 $\pm 20\%$ 안쪽의 차이를 못 잡는다는 뜻이다.

왜 정찰 회차가 덜 머지되나. 클로드 회차의 결말을 나눠 보면 baseline 51건은 머지·릴리즈 65%, CI 없음·실패 18%, 보호 경로 8% 인데 no-scout 20건은 머지 95%, 검토 보류 5% 이고 보호 경로·CI 막힘이 한 건도 없다. 과제 제목을 읽으면 차이가 드러난다 — 정찰은 "gofmt 드리프트 정리 + CI 에 게이트 추가" 처럼 구조적인 일을, 구현자는 "공유 리스트 조회 실패가 '없습니다' 로 감춰지던 문제" 처럼 좁은 결함을 고른다. 자기평가 가치는 3.4 대 3.2 로 거의 같다. 즉 정찰이 고르는 일이 덜 값져서가 아니라 **이 파이프라인에서 끝까지 갈 수 없는 일**이어서 머지율이 낮다. 표본이 작아 (보호 경로 4/51 대 0/20) 기전 자체는 확정이 아니다.

고치는 방법은 정찰을 없애는 것이 아니라 정찰에게 머지 가능성을 알려 주는 것이다 — 보호 경로를 건드리는 일은 그것이 이 과제의 핵심일 때만 고르고, CI 가 없는 저장소에서는 CI 추가 자체를 과제로 고르게 하면 된다. **이 변경은 일부러 미뤘다**. 지금 넣으면 실험 중인 처치(정찰)를 중간에 바꾸는 것이고, 하필 RQ1 이 가장 흥미로운 자리다. 변경안과 확인 방법은 `state/after-experiment.md` 에 적어 두었고, 기한이 지나면 헬스체크가 그 파일을 상기시킨다.

codex-critic arm 은 2026-09-26 에 중단했다. 사전에 정한 중단 규칙은 없었으므로 이것은 사후 판단이고, 그래서 여기 적는다: 처리량 효과가 $p=0.0003$ 으로 확립됐고 남은 일주일에 얻을 정보보다 비용(머지당 \$20.63, 약 \$150~200)이 크다고 봤다. 이미 수집한 31회차는 분석에 그대로 쓰고, 품질 축(머지 뒤 30일)은 이미 머지된 PR 들로 10월에 판독한다. 배정 가중치만 0 으로 바꿨고 다른 arm 은 10-03 까지 그대로 돈다. 중단 사실과 근거는 `state/experiment.json` 의 `stopping_log` 와 arm 별 `stopped` 에 남아 있고, 생성되는 표에도 "중단" 으로 표시된다. 사전 중단 규칙이 없는 실험을 중간에 손대면 그 arm 의 추정치는 낙관적으로 치우칠 수 있다 — 우리가 본 방향(나쁨)과 반대 방향의 편향이므로 결론을 뒤집지는 않지만, 값 자체는 그만큼 덜 믿을 만하다.

7. 타당성에 대한 위협

- **운영자 한 명, 스택 하나**. Go/React/Keycloak 을 공유하는 한 사람의 저장소 46개는 좁은 모집단이다. 특히 캠페인은 그 동질성을 이용한다.
- **짧은 기간, 움직이는 시스템**. 18일 동안 러너가 거의 매일 바뀌었다. 숫자는 시스템의 여러 버전을 합친 것이다.
- **회귀 관찰이 짧다**. "리뷰된 변경이 되돌려진 적 없다" 는 2~48시간 감시와 18일의 이력으로 켜 것이다. §6.8 은 30 일·파일 단위 관점이 훨씬 덜 우호적임을(머지된 변경의 4분의 1이 한 달 안에 사람이 고친 파일에 있음), 그리고 그 관점 자체도 상한일 뿐임을 보여 준다.
- **작은 실험**. E1 은 PR 12건과 대체 모델 하나, E2 는 줄이 아니라 파일 단위 귀속이다. 둘 다 데이터가 쌓이면 다시 돌리도록 만들었다(`bin/exp-cross-critic.sh` , `bin/postmerge.py`).
- **자체 보고 비용**. 정액제에서 모델이 추정한 세션 비용이다.
- **AI 승인에 대한 사람 검토가 얇다**. 운영자는 보호 경로 PR 의 승인을 PR 심사자에게 명시적으로 위임했다. 그 심사자가 대체한 사람만큼 좋은지는 아직 말할 수 없다. 성적표가 "승인 뒤 회귀" 를 세는 이유가 바로 이것이고, 지금은 0 이다.
- **비평가 코드가 같은 모델 계열에서 나온다**. 독립성은 절차적(다른 세션, 다른 도구, diff 만 보는 시야)이지 인식론적이지 않다.
- **엔진이 섞여 있다 (2026-09-25 확인)**. 클로드가 사용량 한도에 걸리면 러너는 같은 프롬프트를 Codex 로 돌린다 (§8). 이 대체는 실험 기간에 드물지 않았다 — 실험 회차의 46~56% 가 Codex 였고, 그 비율이 arm 마다 최대 10%p 달랐다. 대체 회차는 절반이 변경 없이 끝나므로 (171회 중 85회), arm 사이의 차이 가운데 일부는 역할이 아니라 엔진 구성의 차이다. 2026-09-25 부터 회차에 `engine` 을 기록하고, `bin/exp-analyze.py` 는 과거 회차의 엔진을 복원(구현 단계 비용이 비어 있으면 Codex)해 **클로드 회차만 따로 집계한 표**를 함께 낸다. 두 표의 순위는 같지 않다.

- **시스템의 자기 측정이 성공 쪽으로 편향돼 있었다 (2026-09-25 수정).** 승인 스위치는 "아무것도 달라지지 않은 승인 확인"을 기록하지 않으려고 결말이 `merged` 일 때만 회차를 남겼다. 그런데 머지 뒤 릴리즈까지 가면 결말은 `releasing · release-ready` 로 바뀐다 — 머지도 하고 릴리즈도 한 회차만 골라서 기록에서 빠졌다. 그렇게 사라진 회차가 71건이고 증거 디렉터리도 함께 지워졌다. 이 글의 이전 판(9-19)을 포함해 그 전까지의 머지·릴리즈 수는 모두 그만큼 낮게 잡혀 있었다. 56건은 머지 커밋 시각으로 PR 을 복원해 되살렸고(`bin/backfill-approve-runs.py`), 나머지는 시간 창 안에 머지 커밋이 하나로 좁혀지지 않아 두었다. 무인 운영에서 배울 점은 이렇다: **로그를 쓰는 조건이 결과에 따라 달라지면 그 로그로 켜 모든 지표가 같은 방향으로 휜다.** 러너가 자기 성과를 스스로 지우는 동안 아무도 몰랐던 이유는, 그 사실을 알려면 사라진 기록을 봐야 했기 때문이다.

8. 사고와 노하우

아래는 무인 에이전트 러너를 만드는 누구에게든 건넬 목록이다. 항목마다 최소 하루를 허비했다.

8.1 프롬프트가 아니라 배관

1. **while read 루프 안에서 stdin 을 읽는 것은 루프를 삼킨다.** 승인 스위치가 PR 몇 건 뒤 멈췄고(`gh` 가 `stdin` 소비, 09-07), 캠페인 교훈 정제기는 첫 캠페인만 처리했다(`claude -p` 가 `stdin` 소비, 09-17). 루프 안의 모든 자식 프로세스에 `</dev/null`.
2. **스케줄러는 당신의 프로세스를 죽인다.** Windows 작업 스케줄러가 다음 트리거에 WSL 래퍼를 끝내며 회차까지 가져갔다. 10분 넘는 회차가 하루 종일 하나도 끝나지 않았다. 회차는 `setuid` 로 분리하고, 래퍼는 기다리고, 실행 전에 스크립트를 복사해 회차 중의 `git pull` 이 도는 코드를 바꾸지 못하게 한다.
3. **자기 상태 저장소의 쓰기를 직렬화하라.** 두 회차가 동시에 `git pull --rebase` 를 해 남긴 `.git/rebase-merge` 가 이후 모든 동기화를 막았다. `--autostash` 는 회차 기록 한 줄을 조용히 잃었다. 잠금을 잡고, `pull` 전에 `add·commit` 하고, `stash` 없이 재시도한다.
4. **에이전트를 제한하려고 저장소 설정을 쓰지 마라.** 공유 클론의 `git remote set-url --push DISABLED` 는 운영자 자신의 `push` 도 막았다. 프로세스 범위의 `GIT_CONFIG_*` 환경변수를 쓴다.
5. **탭 구분은 직렬화 형식이 아니다.** TSV 의 여러 줄 캠페인 목표가 캠페인을 만료로 닫았고, 빠진 열 하나가 캠페인 넷을 완료로 닫았다. 자유 텍스트는 base64 로, 빈 목록에는 아무것도 하지 않는다.
6. **worktree 는 낫다.** 디렉터리는 사라졌는데 등록이 `locked` 로 남은 `worktree` 는 `worktree add` 를 영원히 실패시킨다. `add` 전마다 `unlock → remove → rm → prune` 으로 리셋하고, 인프라 실패는 프로젝트 실패와 따로 기록한다.
7. **알림 채널도 모의 실행을 알아야 한다.** 모의 실행 스위트는 가짜 GitHub 으로 실제 러너를 돌린다. 그 "simproj 머지" 메시지가 실제 텔레그램에 닿아 침입 의심으로 보고됐다. 모든 결 채널에 격리 플래그가 필요하다.
8. **jq 의 // 는 false 를 없는 값으로 본다.** `false` 로 둔 불리언 시나리오 플래그가 `true` 로 읽혔다. `if .x == false` 로 검사한다.

8.2 게이트와 판단

9. **검증은 러너가, 자기 인증은 금지.** 에이전트가 "테스트 통과" 라고 보고한 회차의 11% 가 러너가 돌린 같은 테스트에 실패했다.
10. **독립 리뷰는 값을 한다.** 검증을 통과한 diff 의 18% 를 diff 만 보는 비평가가 구체적 사유와 함께 거절했다(패널을 언마운트하는 `setPreview(null)`, 원본 문자열에 적용된 소문자 사본의 바이트 오프셋, 항상 거짓인 조건이 지키던 평문 인증 경로). 승인된 273건 중 지금까지 회귀한 것은 없다.

11. "판단할 수 없음" 과 "나쁘다고 판단함" 을 구분하라. 없거나 깨졌거나 예산을 넘긴 리뷰는 보류이지 승인이 아니다.
12. 예산 보류가 실패처럼 보이게 두지 마라. `error` 로 기록된 198회차 중 163회차가 "hold: budget" 이었다. 따로 기록하기 전까지 실패 통계, 캠페인 스트라이크, 사람의 주의를 오염시켰다.
13. 인프라 실패는 자체 결과로 기록하라. 그러지 않으면 깃 `worktree` 가 저장소를 캠페인에서 쫓아낸다.
14. 자율화는 사람 없이 내려갈 수 있어야 한다. 롤백·회귀 시 자동 강등이 있어서 기본 자율화를 `release` 로 올릴 수 있었다.
15. PR 이 아니라 SHA 를 승인하라. 움직이는 브랜치의 라벨은 동의가 아니다.
16. 사람을 대신하는 심사자는 불확실하면 거절해야 한다. 회차 안의 비평은 "불확실하면 승인하고 적어라" 지만, PR 심사자는 그 반대다. 그 뒤에 사람이 없기 때문이다.
17. 판정자의 권고를 그대로 두라. 숙고한 `merge` 를 러너 쪽 위험도 임계값으로 뒤집는 것은 아무도 묻지 않은 두 번째 의견이었다. 운영자가 없었다.

8.3 협업

18. 고르기와 하기를 나누라. 커밋할 수 없는 정찰은 쉬운 과제를 고르지 않는다.
19. 구현자에게 확신 없는 곳을 묻고 비평을 거기로 보내라. 명시적 "검증 못 함" 목록은 우리가 찾은 가장 싹 리뷰 안내였다. 빈 목록 자체가 시험할 주장이다.
20. 의견 충돌에 세 번째 목소리를 주라. 수리가 "지적이 틀렸다" 고 할 때, 두 고정된 입장 사이의 루프는 양쪽을 읽고 코드를 돌려 보는 세션 하나보다 나쁘다.
21. 저장소 프로필을 영속화하라. 세션당 10분의 재독을 하루 마흔 번 하면 세션 하나 분량의 예산이다. 정찰이 유지하고 14일마다 갱신하는 프로필이 그 대부분을 없애고 모든 역할에 같은 그림을 준다.
22. 거절을 규칙으로 정제해 상류에 주입하라. 캠페인 교훈 규칙이 생기기 전에 같은 결함이 두 저장소에서 비싸게 발견됐다. 이후로는 남은 모든 대상의 프롬프트에 그 규칙이 있다.
23. 운영자의 말을 재사용 가능하게 만들라. 반려 사유, 정지 사유, 채팅 지시가 모든 에이전트가 읽는 취향이 된다. 운영자가 "attribution 트레일러 넣지 마" 를 두 번 말할 일은 없어야 한다.
24. 단계마다 알리되 내용으로 중복을 제거하라. 같은 "CI 없음" 메시지가 하루에 열두 번 온 뒤 같은 알림을 여섯 시간 억제하게 됐다. 그 규칙은 다시 코파일럿의 같은 답을 숨겼고, 예외를 두어야 했다.

8.4 사람과의 점점

25. 작업함은 왜, 무엇을 할지를 말해야 한다. 열린 PR 목록은 행동할 수 없다. 보류된 단계, 사유, 증거 링크, 달 라벨이 있는 목록은 할 수 있다.
26. 폰이면 충분하다. 라벨로 승인·반려, `run:` · `stop:` 이슈, 그리고 이제 문장을 작은 고정 동사 집합으로 바꾸는 텔레그램 코파일럿. 동사 목록이 곧 권한 경계다. 코파일럿 세션은 스크립트 하나만 실행할 수 있고 다른 것은 못 한다.
27. 어떤 것은 여전히 사람이 소유해야 한다. 워크플로 파일(비밀값 유출 경로), 결제 코드, 라이선스, 자율화를 올리는 결정. 처리기의 금지 목록이 이것을 부호화하고, 대시보드의 "사람 필요" 열이 그것이 도착하는 곳이다.

9. 관련 연구

이 글은 벤치마크가 아니라 경험 보고다. 네 갈래의 연구와 견주어 자리를 잡는다.

저장소 수준 코딩 에이전트. SWE-bench [1] 가 실제 저장소의 이슈 해결을 표준 과제로 세웠고, SWE-agent [2] 는 모델에게 어떤 명령과 화면을 주는가 — *에이전트·컴퓨터 인터페이스* — 가 모델만큼 중요함을 보였다.

AutoCodeRover [3] 와 Agentless [4] 는 자유로운 에이전트보다 구조화된 파이프라인(위치 찾기 → 수리 → 검증)을 주장했고, 특히 Agentless 는 고정된 3단계로 에이전트 시스템들과 맞먹는 성능을 훨씬 적은 비용에 냈다.

OpenHands [5] 와 RepairAgent [6] 는 같은 공간의 열린 플랫폼과 에이전트다. adev 의 정찰 → 구현 → 검증 구조는 자유 에이전트보다 Agentless 에 가깝다. *회차* 는 고정 파이프라인이고 구현 단계만 긴 지평의 에이전트다. 우리의 기여는 더 나은 해결기가 아니라 패치가 *생긴 뒤*에 일어나는 일 — 리뷰, 머지, 릴리즈, 그리고 무엇이 잘못됐는지의 회계 — 이다.

비판, 정제, 판정. Self-Refine [7] 과 Reflexion [8] 은 모델이 자기 피드백이나 언어 피드백으로 출력을 개선할 수 있음을 보였다. 우리의 수리자와 프로젝트별 교훈은 그 생각의 운영 형태이되, 피드백이 편집할 수 없는 *다른* 세션에서 온다는 점이 다르다. CriticGPT [9] 는 훈련된 LLM 비평가가 모델이 쓴 코드의 버그를 유료 인간 리뷰어보다 더 많이 잡지만 사소한 지적을 지어내기도 함을 발견했다 — 우리 중재자의 동기다. LLM-as-a-judge [10] 는 위치·장황함·자기 강화 편향을 기록했고, Wataoka 외 [11] 는 심판이 낮은 perplexity(익숙한) 텍스트를 선호하는 자기 선호 편향을 정량화했다. adev 의 구현자와 비평가는 같은 모델 계열이므로 §6.6 에서 교차 모델 일치도를 직접 잰다.

다중 에이전트 소프트웨어 프로세스. MetaGPT [12] 와 ChatDev [13] 는 사람의 소프트웨어 역할(PM, 엔지니어, 리뷰어)을 표준 운영 절차를 가진 에이전트로 부호화했고, AutoGen [14] 과 Magentic-One [15] 은 대화·오케스트레이터 기반 프레임워크를 제공한다. 가장 관련 깊은 최근 결과는 *다중 에이전트 LLM 시스템은 왜 실패하는가* [16] 로, MAST 분류는 실패를 시스템 설계, 에이전트 간 불일치, 과제 검증·종료로 귀속시킨다. 우리는 그 분류를 우리 실패를 보는 관점으로 채택했다 (§6.7). Geng 과 Neubig [17] 은 git worktree, 격리 실행, 테스트 기반 통합으로 조정되는 비동기 에이전트를 제안하는데, adev 가 병렬 회차에 쓰는 것과 같은 원시 요소다. Tang 과 Runkler [18], De Oliveira 외 [19] 는 설계 공간을 조사하고 프레임워크 선택을 보고하며, 후자는 대부분의 프레임워크에 에이전트 텔레메트리가 아직 없다고 지적한다 — 우리의 단계 기록, 노트, 성적표가 채우는 자리다.

야생의 에이전트 PR. 20252026년의 실증 연구들은 GitHub에서 에이전트가 쓴 PR에 무슨 일이 일어나는지를 다룬다. Li, Zhang, Hassan [20, 21] 은 AiDev 데이터셋(116,211개 저장소의 에이전트 PR 932,791건)을 공개하고 에이전트가 사람보다 빠르지만 덜 수용됨을 발견했다. Peralta 외 [22] 는 사람이 리뷰한 에이전트 PR 9,799건을 분석해 반려의 35.7% 만이 명백한 에이전트 실패였고, 31.2% 는 워크플로 제약, 33.1% 는 기록된 근거가 없었으며, 수용된 PR의 15.4% 도 리뷰어의 명시적 개입이 필요했음을 보였다. Nachuma 와 Zibran [23] 은 리뷰어 참여가 통합의 가장 강한 상관 요인이고 force push와 큰 diff 가 그것을 줄임을 발견했다. Xia 와 Miller [24] 는 머지 뒤 운명을 추적해 에이전트 기여가 더 많은 수정 유지보수를 필요로 하고 보안·의존성 결함을 더 들여오며, 프로젝트의 무리뷰 비율이 10%p 오를 때마다 유지보수 부담이 약 6% 늘어남을 보였다. Kraishan [25] 은 에이전트별로 다른 되돌림률(6.114.5%)과 긴 리뷰 지연을 발견했다. 이 결과들이 우리 장치 셋의 직접적 동기다. adev 의 모든 보류는 기계가 읽을 수 있는 사유를 가지고(33% 의 "근거 없음"에 대한 답), 처리기는 새 PR 을 여는 대신 PR 안에서 리뷰 피드백에 답하며, §6.8 은 우리 데이터에 30일 머지 뒤 수정 유지보수 측정을 더한다.

Anthropic 의 에이전트 워크플로 패턴 안내 [26] 는 우리가 쓰는 평가자·최적화자와 오케스트레이터·워커 패턴을 이름 짓고 단순하게 시작하라고 권한다. 우리의 이력(§3.1)은 운영 압력 아래 하나의 에이전트에서 그 패턴에 도달한 사례다.

10. 앞으로

- **실험 판독.** RQ1RQ5의 1차 판독은 일말(~~arm 당 약 100200회차~~), 2차는 30일 머지 뒤 결과가 무르익는 10월 중순. 그 뒤 지는 arm 을 버리고 이기는 arm 을 기본 정책으로 올린다.
- **회귀 예측기.** 건드린 파일, 보호 경로, 리뷰 위험도, 테스트 수, 프로젝트 건강으로 각 변경을 자동 머지·처리기·사람 중 어디로 보낼지와 관찰 창 길이를 정한다.
- **더 싼 역할.** 정찰과 비평은 읽기 전용이고 짧다. 더 작은 모델의 후보다. 명부는 자동 풀백이 있는 역할별 모델을 지원하지만 낮은 비용에서의 품질은 아직 재지 않았다.
- **더 긴 기억.** 프로필과 교훈이 회차를 넘는 기억의 첫 두 형태다. 프로젝트별 "최근 비평 셋이 걱정한 것", 캠페인별 "사람이 반려한 것" 이 다음 후보다.
- **두 번째 운영자.** 여기 모든 것은 소유자 한 명을 가정한다. 취향, 자율화, 승인에 신원이 필요해진다.

참고 문헌

1. C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, K. Narasimhan. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR 2024. arXiv:2310.06770.
2. J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, O. Press. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering.* NeurIPS 2024. arXiv:2405.15793.
3. Y. Zhang, H. Ruan, Z. Fan, A. Roychoudhury. *AutoCodeRover: Autonomous Program Improvement.* ISSTA 2024. arXiv:2404.05427.
4. C. S. Xia, Y. Deng, S. Dunn, L. Zhang. *Agentless: Demystifying LLM-based Software Engineering Agents.* FSE 2025 (Proc. ACM Softw. Eng.). arXiv:2407.01489.
5. X. Wang et al. *OpenHands: An Open Platform for AI Software Developers as Generalist Agents.* ICLR 2025. arXiv:2407.16741.
6. I. Bouzenia, P. Devanbu, M. Pradel. *RepairAgent: An Autonomous, LLM-Based Agent for Program Repair.* ICSE 2025. arXiv:2403.17134.
7. A. Madaan et al. *Self-Refine: Iterative Refinement with Self-Feedback.* NeurIPS 2023. arXiv:2303.17651.
8. N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao. *Reflexion: Language Agents with Verbal Reinforcement Learning.* NeurIPS 2023. arXiv:2303.11366.
9. N. McAleese, R. M. Pokorny, J. F. Cerón Uribe, E. Nitishinskaya, M. Trebacz, J. Leike. *LLM Critics Help Catch LLM Bugs.* arXiv:2407.00215, 2024.
10. L. Zheng et al. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena.* NeurIPS 2023 Datasets & Benchmarks. arXiv:2306.05685.
11. K. Wataoka, T. Takahashi, R. Ri. *Self-Preference Bias in LLM-as-a-Judge.* NeurIPS 2024 Safe Generative AI Workshop. arXiv:2410.21819.
12. S. Hong et al. *MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework.* ICLR 2024 (oral). arXiv:2308.00352.
13. C. Qian et al. *ChatDev: Communicative Agents for Software Development.* ACL 2024. arXiv:2307.07924.
14. Q. Wu et al. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation.* arXiv:2308.08155, 2023 (COLM 2024).

15. A. Fournay et al. *Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks*. arXiv:2411.04468, 2024.
16. M. Cemri, M. Z. Pan, S. Yang, et al. *Why Do Multi-Agent LLM Systems Fail?* NeurIPS 2025. arXiv:2503.13657.
17. J. Geng, G. Neubig. *Effective Strategies for Asynchronous Software Engineering Agents*. arXiv:2603.21489, 2026.
18. Y. Tang, T. Runkler. *LLM-Based Agentic Systems for Software Engineering: Challenges and Opportunities*. GenSE 2026 workshop. arXiv:2601.09822.
19. M. C. S. De Oliveira, M. O. Ibiyo, M. Gianrusso, C. Di Sipio, D. Di Ruscio, P. T. Nguyen. *Developing LLM-based Multi-Agent Systems in Software Engineering: A Mixed-Method Experience Report*. arXiv:2608.11965, 2026.
20. H. Li, H. Zhang, A. E. Hassan. *The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering*. arXiv:2507.15003, 2025.
21. H. Li, H. Zhang, A. E. Hassan. *AIDev: Studying AI Coding Agents on GitHub*. MSR 2026. arXiv:2602.09185.
22. S. R. O. Peralta, F. Hoshi, H. Washizaki, N. Ubayashi, et al. *Why Are Agentic Pull Requests Merged or Rejected? An Empirical Study*. arXiv:2605.22534, 2026.
23. C. Nachuma, M. Zibran. *When AI Teammates Meet Code Review: Collaboration Signals Shaping the Integration of Agent-Authored Pull Requests*. arXiv:2602.19441, 2026.
24. C. S. Xia, C. Miller. *Do These Violent Delights Have Violent Ends? Measuring the Post-Merge Fate of Agentic Code*. arXiv:2607.09902, 2026.
25. O. Kraishan. *Not All Agents Are Equal: Code Quality and Post-Merge Maintenance Across Five Autonomous Coding Agents in the Wild*. arXiv:2609.17598, 2026.
26. Anthropic. *Building Effective Agents*. Engineering blog, December 2024. <https://www.anthropic.com/engineering/building-effective-agents>

부록 A. 회차 기록 (발체)

```
{ "ts": "2026-09-17T11:47:03+09:00", "project": "weekly", "outcome": "release-ready",  
  "result": "fix-round: merged https://github.com/hkjang/weekly/pull/17, released v0.303.0",  
  "run_id": "2026-09-17-110259-weekly-improve", "base_sha": "04fdb9c...", "head_sha": "58851d1...",  
  "stages": { "base": { "state": "pinned" }, "verify": { "state": "passed", "reason": "검증 7개 통과 (auto)" },  
             "pr": { "state": "created" }, "review": { "state": "approved", "reason": "리뷰 승인 (risk=low)" },  
             "ci": { "state": "passed" }, "merge": { "state": "done" }, "release": { "state": "published" },  
             "assets": { "state": "verified" } },  
  "files": 6, "additions": 212, "deletions": 31, "tests": 2 }
```

부록 B. 기본값

설정	기본값
단계별 예산 (USD)	정찰 2 · 구현 8 (캠페인 12~18) · 비평 4 · 수리 6 · 중재 3 · 릴리즈 10 · 자산 15
일일 상한	비용 \$500 · 회차 100 · 릴리즈 60 (캠페인·처리기는 자체 예산)
제한 시간	구현 45분 · 비평 15분 · 정찰 10분 · 릴리즈/자산 60분
자율화	<code>release</code> ; 롤백·회귀 시 한 단계 강등
보호 경로	<code>.github/workflows/</code> , <code>migrations/</code> , <code>'auth</code>
PR 처리기	매시간 · 한 번에 6건 · 하루 \$150 · PR 당 2회 · 금지: 워크플로·결제·비밀값·LICENSE
캠페인 스트라이크	오류 3회 → 막힘; <code>infra-error / usage-limit</code> 은 세지 않음

부록 C. 용어

회차 — 한 저장소에 대한 러너 작업 한 단위. **과제서** — 정찰의 과제 기술. **회차 노트** — 모든 역할이 덧붙이는 회차별 파일. **보호 경로** — PR 을 사람 검토로 보류시키는 파일 정규식 목록. **자율화** — 러너가 무인으로 해도 되는 일의 프로젝트 별 상한. **캠페인** — 표준과 참조 구현을 가진 유한한 저장소 간 일감. **PR 처리기** — 멈춘 PR 을 되살리는 매시간 프로세스. **기록자** — 거절·회귀를 규칙으로 정제하는 오프라인 작업. **코파일럿** — 문장을 `ops.sh` 동사로 바꾸는 텔레그램 인터페이스.

데이터: `docs/data/runs.jsonl` , `docs/data/usage.jsonl` , `state/lessons.jsonl` , `state/shepherd.jsonl` , `state/campaigns.json` . *코드:* `bin/run.sh` , `agents/` , `AGENTS.md` . *대시보드:* <https://hkjang.github.io/aidev/>